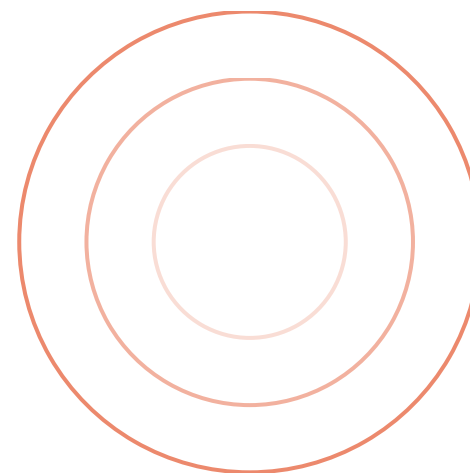


CLOUDLET'S SERVERLESS

From Knative to the Portal

What we built, and why.

Team review · September 2026



TODAY

What we'll talk about

- What is serverless computing
- What is Knative and OpenShift Serverless Operator
- The API: active-active Knative wrapper
- Functions: buildpacks, kpack, build controller
- The portal and cloudlet-apis
- Where we go next

26

slides · six topics · one platform

What is serverless?

- You bring code, not servers
- Scale to zero when idle
- Scale out when traffic arrives
- Pay for requests, not hours
- Nothing to patch, nothing to plan

0 → N → 0

replicas follow traffic

Serverless: the trade-offs

	PRO	CON
Cost	Pay only when it runs	Bills spike under load
Ops	No servers to patch	Runtime is not yours
Scale	Up and down by itself	Cold start on first hit
Speed	A function in minutes	Platform lock-in
Fit	Bursty, event-driven	Long-running jobs, state

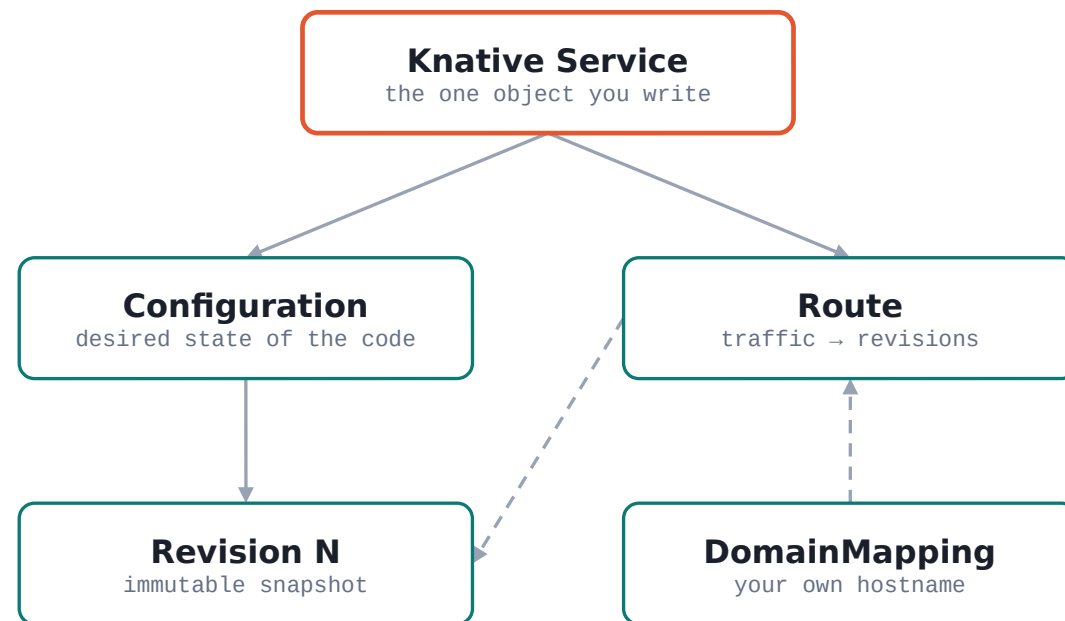
Knative, from what we already run

- We know Deployment, Service, Ingress, HPA
- Knative bundles them into one object
- Adds revisions, routing and scale-to-zero
- Runs on any Kubernetes, ours is OpenShift

NEED	PLAIN KUBERNETES	KNATIVE
Run the code	Deployment	Revision
Reach it	Service + Ingress	Route
Scale it	HPA, never to zero	KPA, down to zero
Change it	Rolling update	New Revision, old kept
Own hostname	Ingress rule	DomainMapping

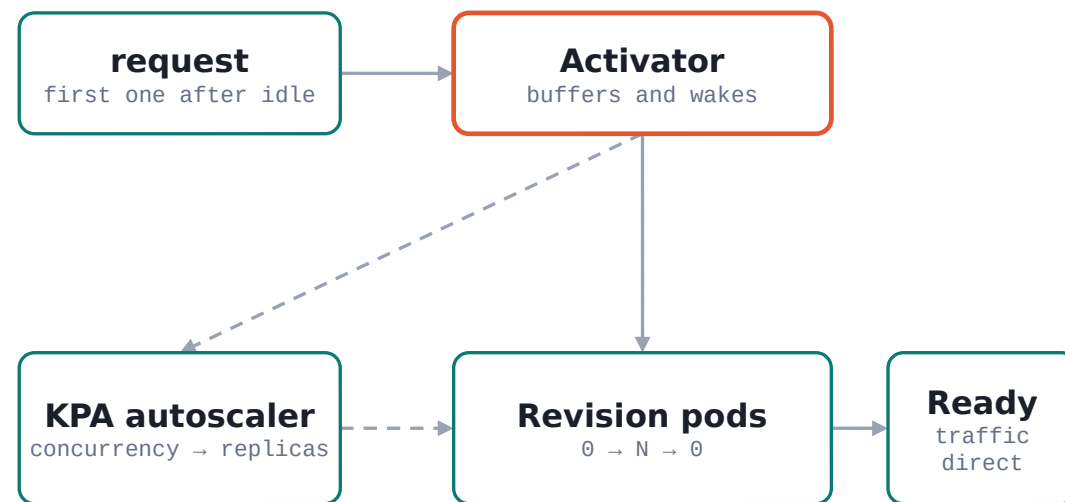
One object: the Knative Service

- The Service owns Configuration and Route
- Configuration holds the desired code
- Every change is an immutable Revision
- Route sends traffic to revisions
- DomainMapping gives it your hostname



Scaling to zero, and back

- KPA watches concurrency per revision
- An idle revision scales to zero
- The Activator holds the first request
- Pods come up, the request continues
- Bursts scale out to maxScale



The OpenShift Serverless Operator

- Red Hat's supported Knative distribution
- One KnativeServing custom resource
- OLM installs it and upgrades it
- Kourier ingress, Routes created for you
- Catalog mirrors with oc-mirror

Knative Serving

Service · Revision · Route ·
DomainMapping

KnativeServing CR

one document

Serverless Operator

reconciles, upgrades

Operator Lifecycle Manager

Subscription → catalog

OpenShift

one cluster per region

Why the Operator

- Every alternative made us own more
- We never create a Route ourselves
- Airgap needs one supported mirror path

	UPSTREAM KNATIVE	SERVERLESS OPERATOR
Upgrades	YAML we maintain	OLM, from the catalog
Ingress	Run it ourselves	Kourier, managed
Routes, TLS	Hand-made, RBAC	Created for us
Airgap	Image by image	oc-mirror, once
Support	Community	Red Hat

The API

One HTTP call, a workload on every cluster.

One call, every cluster

- Functions from git, containers from images
- Validate now, answer 202, poll status
- Closed status vocabulary, published on /info
- FastAPI, Pydantic, cloudlet-apis
- One Helm chart, rendered per region

2

offerings

202

every write

0

kubectl for users

2

regions per deploy

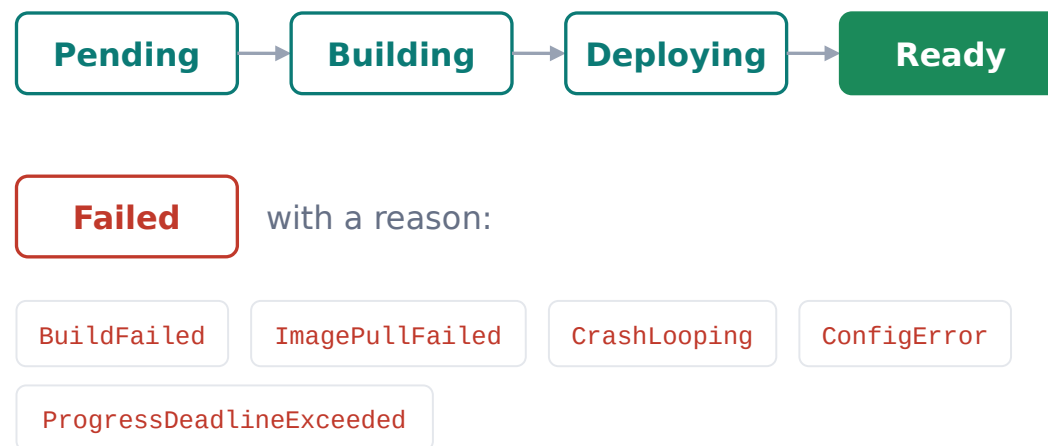
The endpoint structure

- 23 endpoints under one base path
- The group is in the path
- OIDC token or admin key
- Streams use short-lived tickets

```
/api/serverless/v1  
  /groups/{group}/functions  
    POST · GET · PUT · DELETE  
    /{name}/build · /stats · /pods · /logs  
  /groups/{group}/containers → /pull  
  /functions/info · /containers/info  
  /stream-tickets
```

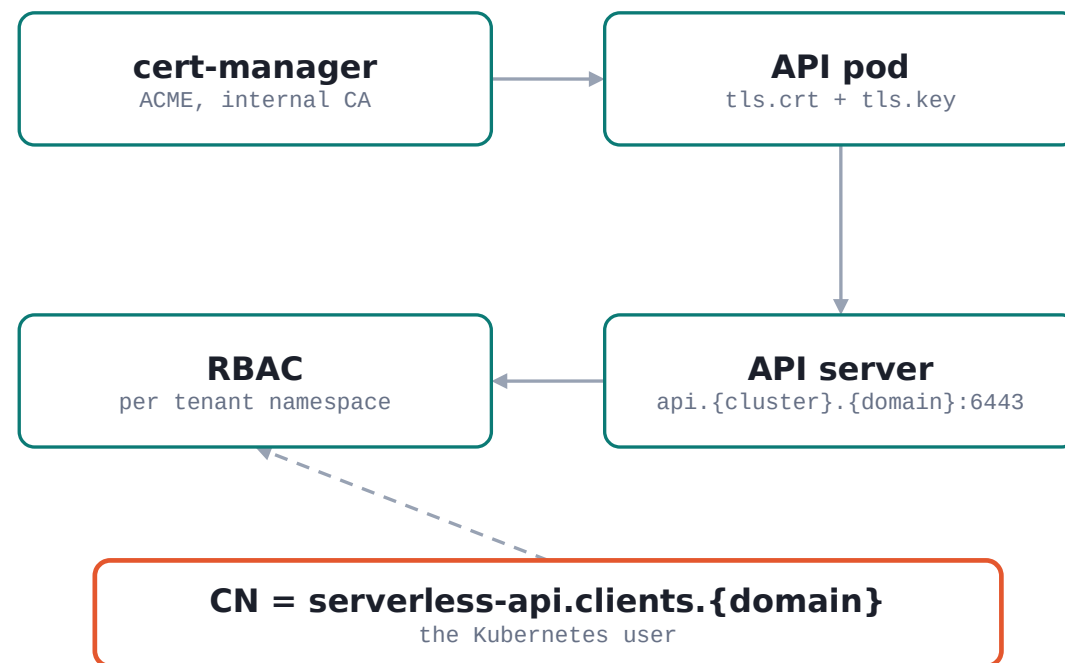
Anatomy of a workload

- Identity: name, hostname, regions
- Source: git repo or image
- Runtime: env, files, port, size
- Scaling: min, max, metric, target
- Response: same shape, secrets redacted



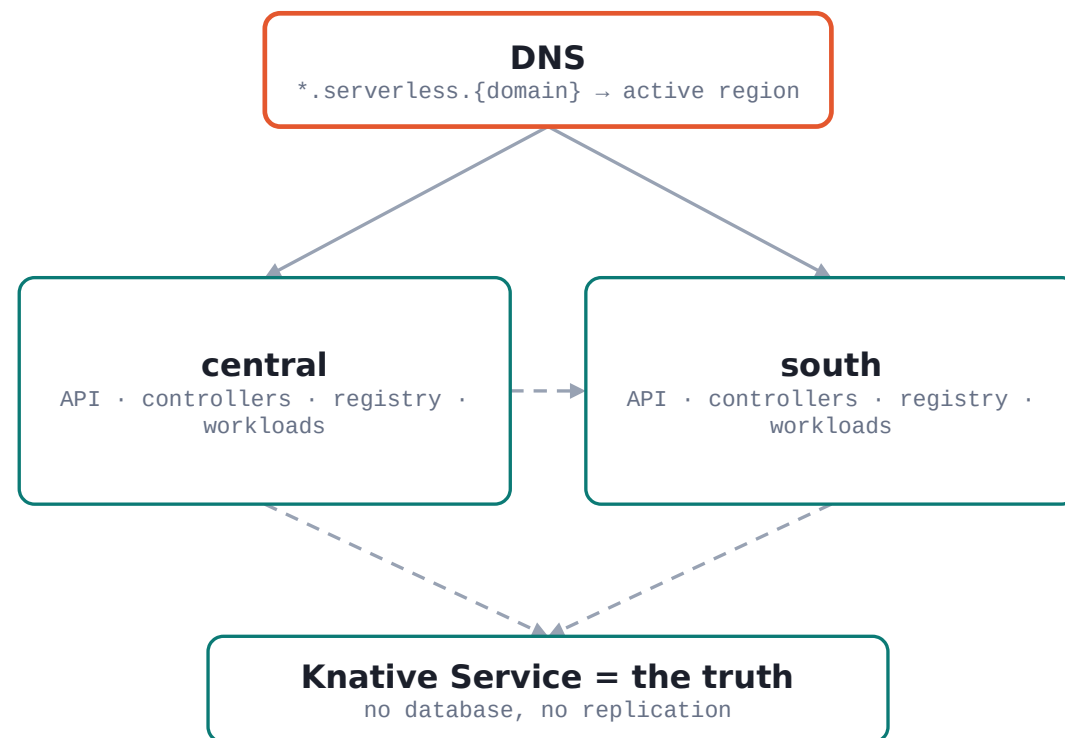
Talking to the cluster

- Client certificate mTLS, always
- No kubeconfig, no service-account token
- The certificate's CN is the user
- Address derived from the region name
- Every write is server-side apply



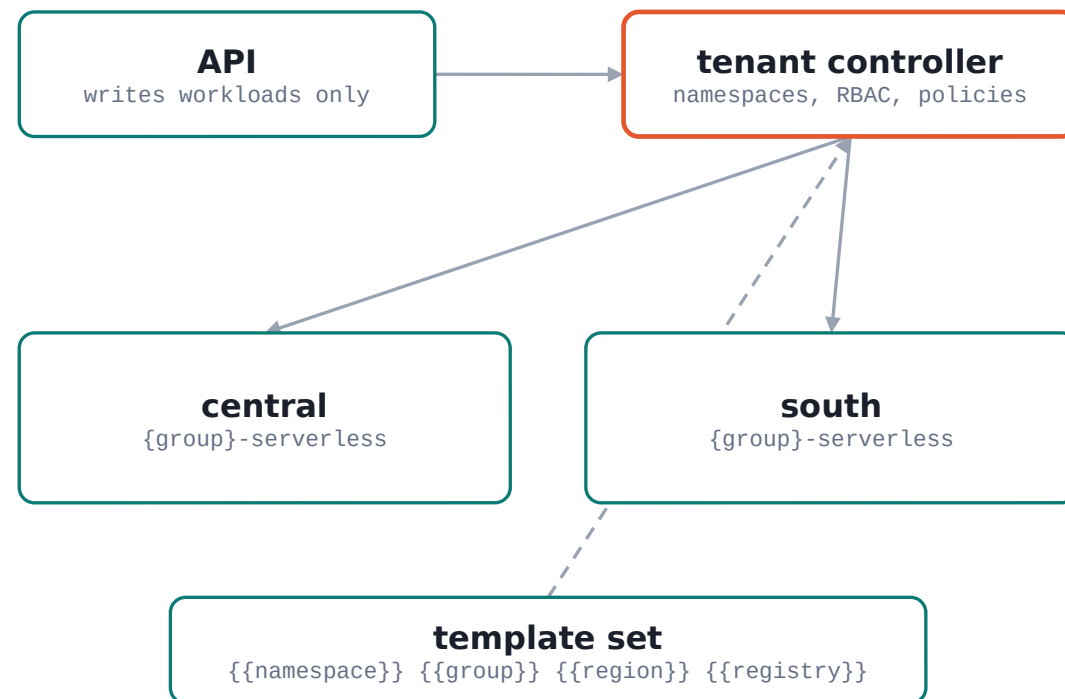
Active/active, two regions

- The API runs in both regions
- One deploy fans out to both
- No leader election, deterministic specs
- A region builds what it runs
- State lives in the Knative Service



The tenant controller

- A namespace per SSO group
- Separate process, separate certificate
- The API never creates namespaces
- Provision before every deploy, fail closed
- Reconcile the local cluster only
- Garbage-collect empty namespaces, opt-in

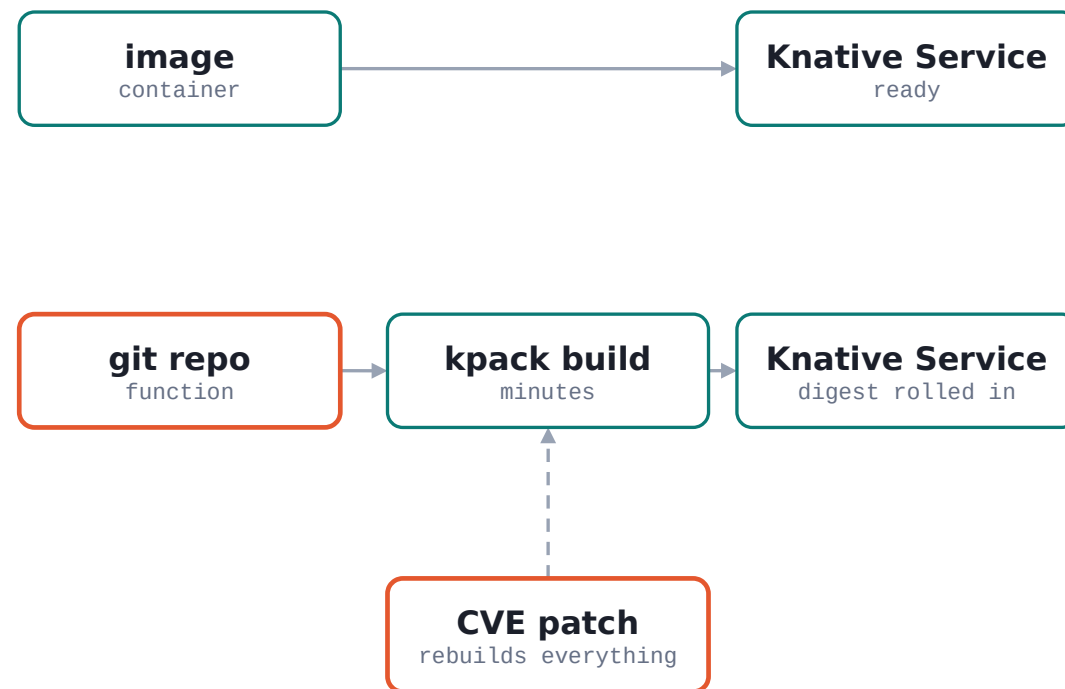


Functions & builds

'Here is my repo' hides a build.

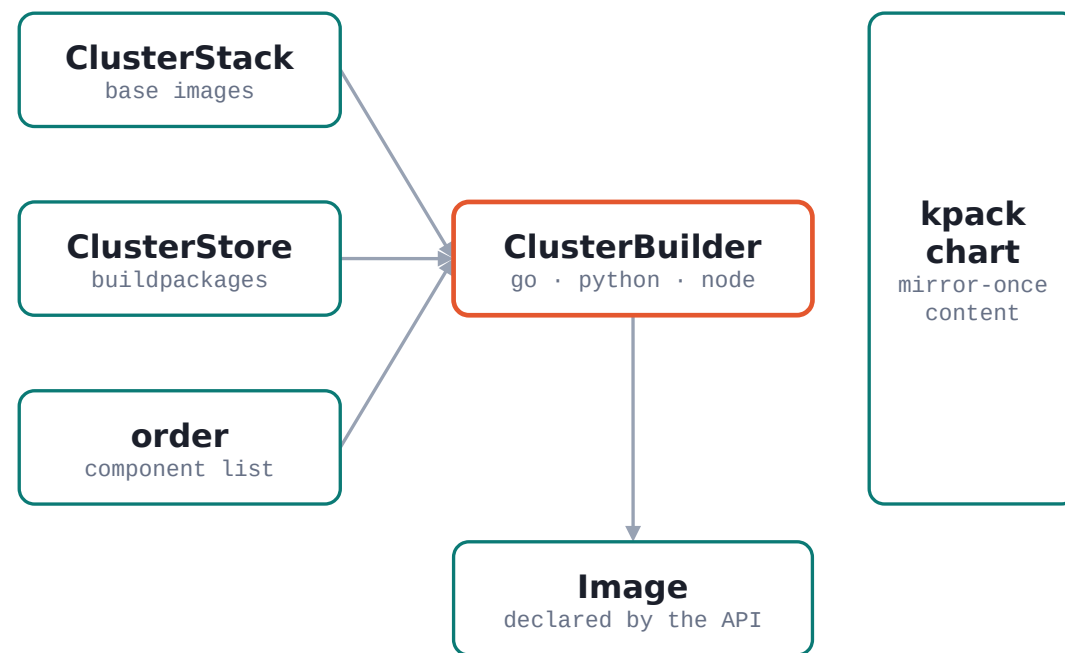
Why functions are different

- Container: run this image
- Function: here is my repository
- A build takes minutes, not milliseconds
- A build is retried and logged
- Rebuilds fire on CVE patches, unasked



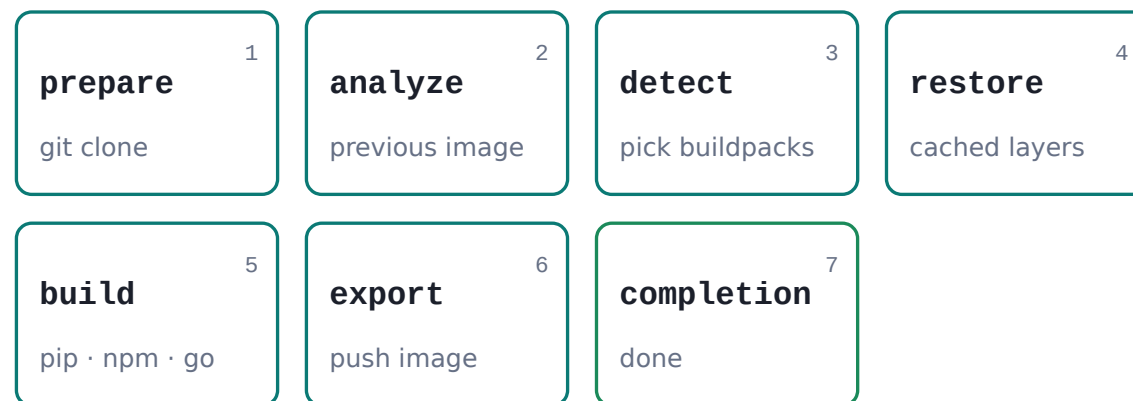
Buildpacks and kpack

- No Dockerfile, buildpacks detect the language
- Rebase: patch the base without rebuilding
- kpack: Image, Build, Pod
- Our kpack repo is a Helm chart
- Airgap mirror scripts included
- ClusterBuilders: go, python, node



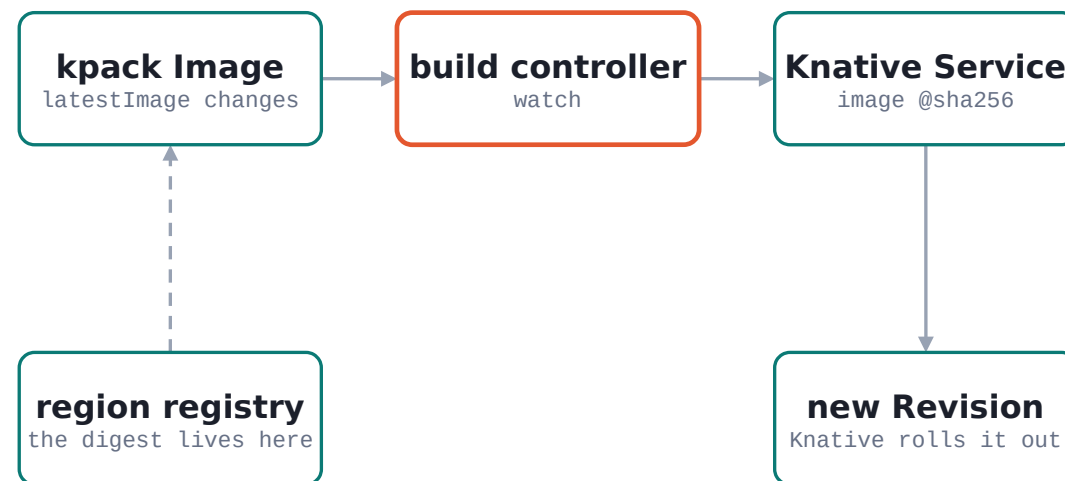
A build, step by step

- Image → SourceResolver → Build → Pod
- Each phase is its own container
- Each phase has its own log
- Pushed to this region's registry
- Five triggers; two need nobody



The build controller

- Watches Image.status.latestImage
- Applies the digest onto the Service
- Both ends stay in one region
- Only ever writes digests
- Ships with no web stack

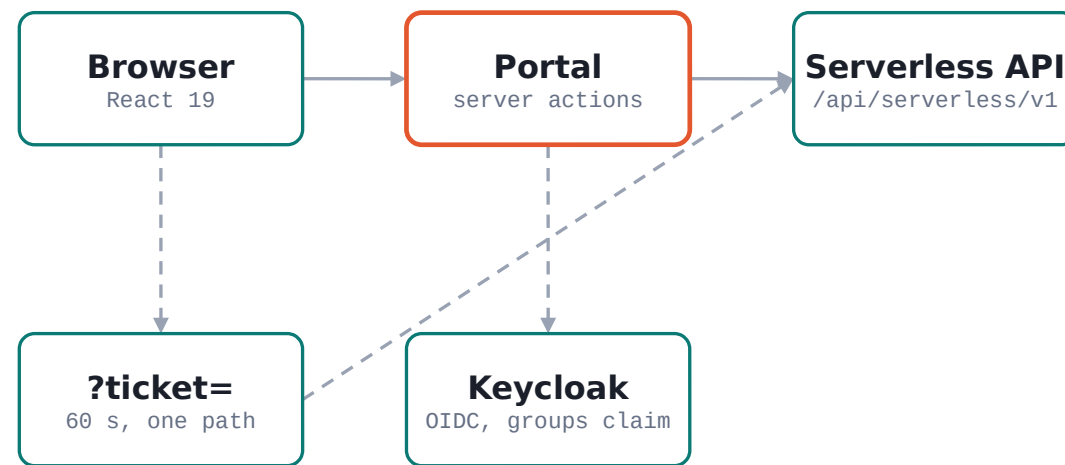


Portal & the future

A console, and a library for the next API.

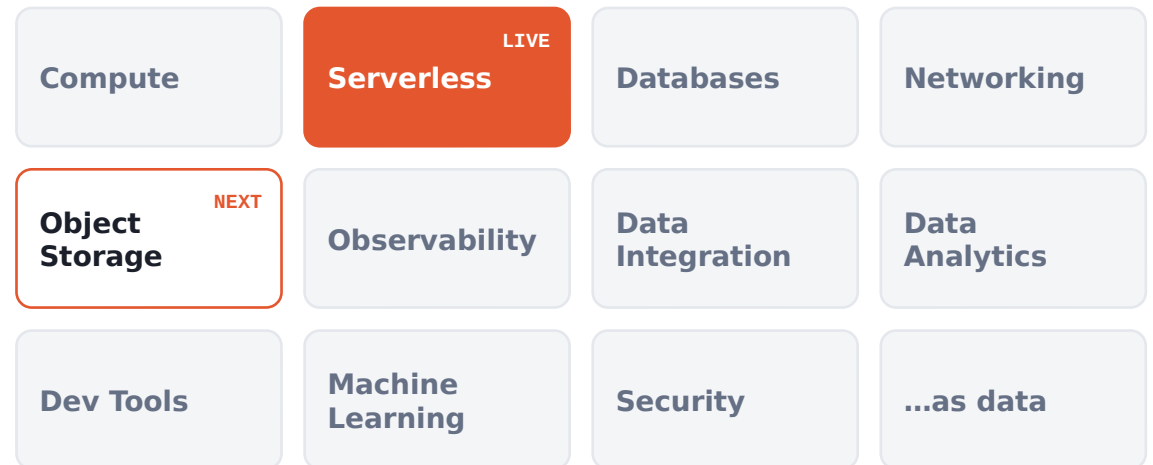
How the portal wires it

- Next.js 16, server components
- Keycloak token stays server-side
- The SSO group is the project
- Forms are built from /info
- Live logs over SSE tickets
- Airgap-native: no CDN, no fonts



A console, not a Serverless UI

- Serverless is live, Storage is next
- An offering is data, not code
- The shell is already generic
- Next: shared layer, generated client



cloudlet-apis: start at step five

- Extracted from the Serverless API
- core, web and auth extras
- Layering enforced by tests
- A new API wires in five steps
- Same group rules everywhere

[auth]

OIDC · admin key · token proxy ·
tickets

[web]

health · error envelope · offline docs

core

errors · names · logging · request id

Code in, a running service out.

4

repositories

3

services

23

endpoints

2

regions

3

runtimes

0

kubectl

- Operator owns Knative; we own the contract
- Deterministic specs, so no leader
- Builds declared, watched, rolled in
- The second product is cheaper

**Thank you.
Questions?**